

Home

Library

Profile

Stories

Stats

Following

Dev Genius

Python in Plain En...

The Useful Tech

Generative AI

Level Up Coding

AI Mind

Top Python Librari...

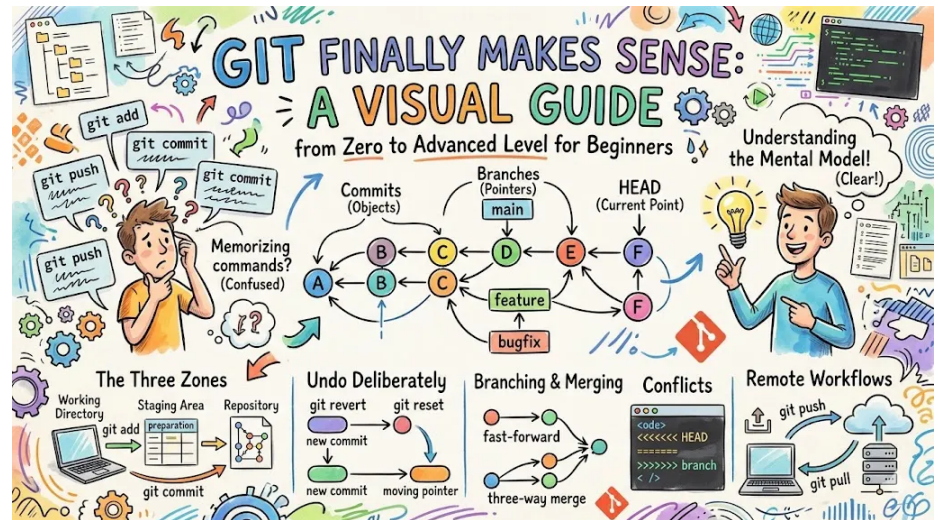
Deep concept

AI Quick Tips

Realworld AI Use C...

More

AI Engineering Simplified



Member-only story

Git Made Easy: A Visual Guide from Zero to Advanced (For Beginners)

Every concept is explained visually, from basics to the advanced level, especially for beginners, to be able to visualize Git.



Divy Yadav

Follow

14 min read · Apr 15, 2026



109



You know that feeling when you type a git command and genuinely aren't sure what just happened?

Most people learn Git by memorizing commands. `git add`, `git commit`, `git push`.

They follow tutorials, things mostly work, and then one day something breaks and they have no idea why or how to fix it without deleting the repo and starting over.

The reason Git feels fragile is not the commands. It is the mental model.

Once you understand Git as a graph, every command becomes obvious. Every “undo” option becomes a deliberate choice. Every branch becomes a simple pointer rather than a mysterious copy of your code.

This article builds that model from scratch.

Foundations first, commands second, internals in the middle, where they make the most sense. If you read this once and actually follow the diagrams, Git will feel different from this point forward.

Part 1: How Git Actually Stores Data

Before any commands, let's talk about what Git is doing under the hood. Not theory, just the thing that makes every other concept make sense.

Every file in your project, every version of every file, every commit message, every directory structure, all of it is stored as four types of objects inside the `.git/objects` folder.

THE FOUR OBJECTS GIT KNOWS ABOUT

```
Blob    = a file's content (just the
bytes, no filename)
Tree    = a directory (maps names to blobs
and other trees)
Commit  = a snapshot (tree + author +
message + parent hash)
Tag     = a named, permanent pointer to a
commit
```

Here is what a single commit actually looks like inside `.git`:

COMMIT OBJECT

```
tree    4b825dc642cb6eb9a060e54bf8d69288fbee4904
parent  3a7f9c21e2b...
author  Divy <divy@email.com> 1712345678 +0530
committer Divy <divy@email.com> 1712345678 +0530
```

Add user authentication

TREE OBJECT (pointed to by commit)

```
100644 blob a3f9c21... main.py
100644 blob b2d7e44... README.md
040000 tree c4e8f55... src/
```

A hash of its contents identifies each object. Change one byte of a file, and you get a completely different hash. This is why Git is trustworthy: it is impossible to silently change history without the hashes not matching up.

The `.git` folder holds everything:

```
.git/
  objects/      ← all blobs, trees, commits, tags
  refs/
    heads/      ← your local branches (just files)
    remotes/    ← remote tracking branches
  HEAD          ← one line: which branch you're on
  index         ← the staging area
  config        ← repository-level settings
```

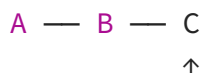
A branch is literally just a text file in `.git/refs/heads/` containing a commit hash. That is it. Creating a branch writes one small

file. Deleting a branch removes it. Branches are cheap because they are pointers, not copies.

Part 2: The Graph Model

Now that you know commits are objects and branches are pointers, the graph makes complete sense.

Start simple:





```
graph TD; main[main] --> HEAD[HEAD];
```

Each letter is a commit object. The arrows mean “this commit’s parent is that one.” `main` is the branch pointer. `HEAD` is where you currently are.

When you make a new commit:

Before:

A — B — C

↑

main, HEAD

After `git commit`:

A — B — C — D

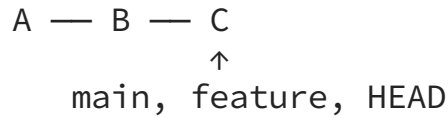
↑

main, HEAD

Git creates commit D, sets its parent to C, then moves the `main` pointer to D. `HEAD` follows because `HEAD` points to the branch, and the branch has moved.

When you create a branch:

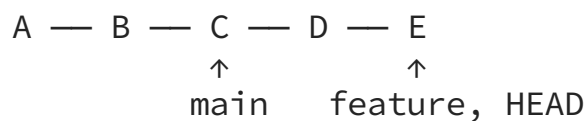
```
git switch -c feature
```



No files changed. No copy was made. A new pointer `feature` was created pointing at the same commit C. HEAD now points to `feature`.

When you commit on the new branch:

```
git commit
```



`feature` moved to E. `main` stayed at C. This is the isolation that makes branching useful.

Hold this graph in your head for everything that follows.

Part 3: The Three Zones

There are three places your code can live at any moment:

```
Working Directory → Staging Area → Repository  
(your disk)         (the index)    (the graph)
```

Working Directory is your project folder. Any file you can open and edit. Git watches this but does not control it.

Staging Area (also called the index) is where you

assemble the next commit. You copy specific changes here before committing. Think of it as a preparation table.

Repository is the commit graph. Once a snapshot is committed here, it is permanent and safe.

Why does staging exist?

Imagine you spent the afternoon fixing a bug in `auth.py` and adding a feature in `profile.py`. You want two commits with clear messages, not one commit that says "misc work."

WORKING DIRECTORY STATE:

```
auth.py      ← bug fix
profile.py   ← new feature
config.py    ← updated constants
```

YOUR GOAL:

```
Commit 1: "Fix null pointer in auth
flow"    ← just auth.py
Commit 2: "Add user profile page"
← just profile.py
Commit 3: "Update staging config"
← just config.py
```

Staging lets you pick exactly which changes go into each commit, regardless of what you worked on together. You are not committing your whole working directory. You are committing the staging area.

Part 4: The Core Commands, Foundation First

Setting up

```
git init # sta
git clone https://github.com/user/repo.git # cop
```

```
git config --global user.name "Your Name"
git config --global user.email "you@email.com"
```

`git init` creates the `.git` folder. That hidden folder is Git. Delete it, and all history is gone, but your files remain.

`git clone` downloads the full object database and creates the `origin` remote connection automatically.

Checking what's happening

```
git status      # see what changed, what's staged
git diff        # working directory vs staging area
git diff --staged # staging area vs last commit
```

`git status` tells you which zone each file is in:

- **Untracked:** new file Git has never seen
- **Modified:** known file that changed
- **Staged:** change ready to be committed

Moving changes through the three zones

```
git add filename.py  # stage one file
git add .            # stage everything
git add -p           # stage specific chunks in
```

`git add -p` is genuinely powerful. It walks you through each change chunk by chunk, letting you decide which parts to stage. This is how you separate one afternoon of work into multiple clean commits.

```
git commit -m "Your message here"
git commit --amend      # rewrite the most recent
```

Write commit messages as documentation for your future self. “Fix bug” is useless in six months. “Fix null pointer when user account has no email address” tells you exactly what happened and why.

`git commit --amend` Rewrites the last commit. Only use this before pushing and amending changes the hash, which breaks anyone who already has the old version.

Reading history

```
git log      # full history
git log --oneline  # compact, one line
```

```
git log --oneline --graph --all      # visual graph
git show a3f9c21                     # inspect a commit
```

The `--graph --all` combination draws the actual commit graph:

```
* e4a91bc (HEAD -> main) Add profile feature
* 3c7f22d Fix auth null pointer
| * 8b1d445 (feature/notifications) Add email system
|/
* 5a2e109 Initial setup
```

Read this like the diagrams above. `*` is a commit node. The `|` and `/` show diverging and reconnecting branches. This is your map.

Blaming and tracing changes

```
git blame filename.py
```

Shows every line of a file annotated with who last changed it and when:

```
a3f9c21 (Divy 2026-03-15 09:22:31) def authentica
b2d7e44 (Priya 2026-03-12 14:55:09)     if user.e
3c7f22d (Divy 2026-03-18 11:30:00)     raise
```

Use this when you want to understand why a line exists, not to assign blame. It answers “who wrote this and in what context.”

Part 5: Understanding Undo in Git

This is where almost every beginner gets confused. Git has multiple undo mechanisms because “undo” means different things depending on what you want to preserve.

The most important thing to understand first:

Now, each one in detail.

git restore: working directory and staging area,

```
git restore --staged main.py    # unstage, keep w  
git restore main.py            # discard working
```

`--staged` remove the file from staging. Your change is still in the working directory, just not queued for the next commit.

Without `--staged`, The change is thrown away. The file goes back to the last committed state.

This cannot be undone.

git reset: move the branch pointer

`git reset` moves the current branch pointer to a different commit. Everything after that commit is either unstaged, discarded, or kept staged, depending on the flag.

CURRENT STATE:

```
A — B — C — D — E
                        ↑
                    main, HEAD
```

After `git reset HEAD~2`:

```
A — B — C
            ↑
        main, HEAD
```

Commits D and E are no longer on main.
(They still exist as orphaned objects temporarily.)

The three flags control what happens to the changes from D and E:

```
--soft    → changes from D and E are kept staged  
--mixed   → changes from D and E are unstaged (default)  
--hard    → changes from D and E are discarded entirely
```

Use reset only on commits that have not been pushed. If D and E were already on the remote and you reset, your local history and the remote's history diverge. Teammates who pulled D and E now have different history than you.
git revert: add an undo commit

```
git revert D
```

Instead of moving a pointer backwards, **revert creates a new commit that undoes the changes from a specific commit.**

BEFORE:

```
A — B — C — D — E
                      ↑
                  main, HEAD
```

AFTER `git revert D`:

```
A — B — C — D — E — D'
                      ↑
                  main, HEAD
```

D' applies the inverse of D's changes.

History is preserved. D is still in the graph. D' undoes its effect. This is why revert is safe for shared branches: you are not rewriting history, you are adding to it.

git checkout: navigate the graph

```
git checkout branchname      # switch to a branch
git checkout a3f9c21         # detach HEAD to a commit
git checkout -b new-branch    # create branch and switch to it
```

`git checkout` is one of Git's oldest commands and does a few different things depending on what you pass it. Newer Git recommends `git`

`switch` for changing branches and `git restore` for file operations, but you will see `checkout` it everywhere in older docs and tutorials.

Detached HEAD happens when you check out a specific commit hash:

```
git checkout a3f9c21
```

```
A — B — C — D — E
                ↑
            HEAD (not pointing to any
branch)
            main → E (still here)
```

You can look around, run the code, inspect files. But if you make a commit here, it is not attached to any branch. When you switch back to main, that commit becomes an orphan and will eventually be garbage collected.

To keep work done in detached HEAD:

```
git switch -c recovery-branch # creates a branch
```

git rebase: replay commits on a new base

```
git rebase main
```

Rebase takes your commits and replays them on top of a different base commit. The result is a linear history as if you had started your branch from the current tip of main.

BEFORE REBASE:

```
A — B — C — F      (main)
                \
                D — E      (feature, HEAD)
```

```
AFTER git rebase main (from feature
branch):
A — B — C — F          (main)
                \
                D' — E'   (feature, HEAD)
```

D and E become D' and E': new commits with new hashes, same changes, but now based on F instead of C.

When to use rebase:

- Before merging your feature branch, to keep history linear
- To incorporate recent changes from main without a merge commit

When not to use rebase:

- On any branch that other people are using. Rebase rewrites hashes. Anyone with the old commits will have divergent history.

The golden rule: never rebase a shared branch.

The full comparison

The key question: Does the commit already exist on a shared remote branch?

- **Yes:** use `revert` or `checkout`. Never reset or rebase.
- **No (local only):** you can use any of them.

Part 6: Branching and Merging

```
git branch feature/login      # create branch
git switch feature/login      # move to it
git switch -c feature/login    # create and move
git branch -d feature/login    # delete after
git branch -D feature/login    # force delete
```

Merging:

```
git switch main
git merge feature/login
```

Two outcomes based on history:

Fast-forward merge (main has not moved since you branched):

BEFORE:

```
A — B — C          (main)
      \
      D — E    (feature/login, HEAD)
```

AFTER merge (fast-forward):

```
A — B — C — D — E
                ↑
            main (moved forward)
```

Git just moved the main pointer. No merge commit created.

Three-way merge (main has new commits since you branched):

BEFORE:

```
A — B — C — F      (main)
              \
              D — E   (feature/login)
```

AFTER merge:

```
A — B — C — F — M      (main)
              \      /
              D — E
```

M is the merge commit. It has two parents: F and E.

Handling conflicts:

When the same lines changed in both branches,

Git marks the conflict and waits:

```
<<<<<<< HEAD
const message = "Hello";
=====
const message = "Hi there";
>>>>>> feature/login
```

Edit the file to what it should be, remove the conflict markers, then:

```
git add conflicted-file.js
git commit      # completes the merge
```

Part 7: Cherry-Pick

`git cherry-pick` applies the changes from a specific commit onto your current branch without merging the whole branch.

```
git cherry-pick a3f9c21
```

Scenario: You are on `main`. A teammate fixed a critical bug on their feature branch (commit `a3f9c21`). You need that fix now, but their branch is not ready to merge. Cherry-pick lets you bring just that commit to `main`.

BEFORE:

```
main:  A — B — C
                ↑
                HEAD
```

```
feature: A — B — C — D — E
                      (D is the bug fix, E
is unrelated work)
```

```
git cherry-pick D
```

```
AFTER:
```

```
main:    A — B — C — D'
                      ↑
                      HEAD
```

D' has the same changes as D but a different hash.

Cherry-picking is surgical. It does not care about branch history, just the diff from one specific commit.

Common use cases:

- Apply a hotfix to a release branch without merging all of main
- Bring one commit from an abandoned branch
- Apply a bug fix across multiple release branches

Part 8: Remote Workflows

A remote is another copy of the repository.

`origin` is the name automatically given to the remote when you clone.

LOCAL MACHINE

REMOTE ([GitHub](#))

`main` → E
`origin/main` → C
`HEAD` → `main`

`origin/main` → C

`origin/main` in your local repo is Git's cached snapshot of where the remote was last time you synced.

Basic remote commands:

```
git remote -v          # show configured remotes
git push origin main    # send commits to remote
git fetch origin        # download remote updates
git pull               # fetch + merge
git push origin feature/login # push a branch
```

Fetch vs pull:

```
# fetch: download only. you stay on your branch until merge
git fetch origin
git log origin/main    # inspect what changed
git merge origin/main  # manually integrate when ready

# pull: fetch + merge. convenient but less control
git pull
```

Fetching before merging is safer in a team. You see what changed before it lands in your branch.

The full team workflow:

1. `git switch -c feature/my-work`
2. Make changes. Commit regularly.
3. `git fetch origin`

4. `git rebase origin/main` (put your work on top)
5. `git push origin feature/my-work`
6. Open pull request on GitHub
7. After approval and merge:
8. `git switch main && git pull` (get the merged code)
9. `git branch -d feature/my-work` (clean up)

Part 9: The Commands You Will Actually Use Every Day

```
# Orientation (use these constantly)
git status
git log --oneline --graph --all
git diff
git diff --staged

# Staging and committing
git add -p # stage in chunks (the p)
git commit -m "message"

# Branching
git switch -c feature/name
git merge branch-name
git rebase main

# Undoing
git restore --staged file # unstage
git restore file # discard working changes
git reset --soft HEAD~1 # undo commit, keep changes
git reset --mixed HEAD~1 # undo commit, unstage changes
git reset --hard HEAD~1 # undo commit, discard changes
git revert abc1234 # safe undo for shared branches

# Investigating
```

```
git blame filename.py           # who wrote each line
git log --oneline --all         # what exists in the
git reflog                      # every HEAD movement

# Surgical operations
git cherry-pick abc1234         # apply one specific
git stash                      # save work temporarily
git stash pop                   # restore stashed work

# Remote
git fetch origin
git pull
git push origin branch-name
```

Part 10: The Recovery Playbook

“I staged the wrong file”

```
git restore --staged wrongfile.py
```

Change stays in working directory. Nothing lost.

“I committed to main by accident”

```
git branch feature/my-work      # save the commit to
git reset --hard HEAD~1         # move main back
```

```
git switch feature/my-work    # continue working
```

Only if not pushed yet.

“I deleted a commit with git reset and want it back”

```
git reflog  
# Find the commit hash  
git switch -c recovered-branch abc1234
```

Git keeps every HEAD position in the reflog for 90 days. Nothing is truly gone until garbage collection runs.

“I’m in detached HEAD and panicking”

```
git switch -c my-recovery-branch    # attach HEAD
```

Now your commits are safe.

“There are conflicts and I don’t know what to

do”

```
git status      # see which files are conflicted
# open each conflicted file, look for <<<<<< mark
# edit to the correct version, remove all conflicts
git add resolved-file.js
git commit      # or git merge --continue
```

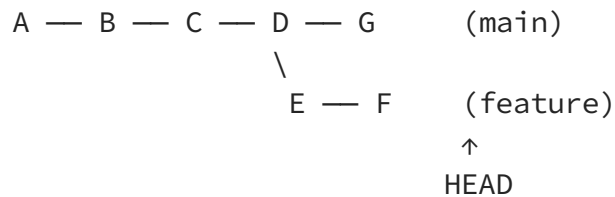
The Model in One Picture

OBJECTS:

- Blob = file **content**
- Tree = directory **structure**
- Commit = snapshot (tree + parent + message)
- Tag = permanent named pointer

REFERENCES:

Branch = mutable pointer to a commit (moves with HEAD)
 HEAD = where you are (points to a branch or commit)
 Remote = cached pointer to a remote commit

THE GRAPH:**THE THREE ZONES:**

Working Dir → Staging → Repository
 (edit here) (add here) (commit here)

COMMAND TO ZONE MAPPING:

git add	moves changes from Working Dir to Staging
git commit	moves Staging to Repository (new commit)
git reset	moves the branch pointer backward
git revert	adds a new node that undoes a previous commit
git rebase	replays nodes onto a different base
git checkout	moves HEAD to a branch or specific commit
git push	copies local nodes to remote
git pull	copies remote nodes to local + merge

Every Git command is an operation on objects and pointers. When a command confuses you, ask: what object is this creating, moving, or deleting, and which pointers are being updated?

Where to Go From Here

You now have the foundation. From here, the path is practice, not more commands.

Build something with Git. Make branches for every feature. Write real commit messages. Intentionally break something and recover it. Use `git log --graph` daily until reading the graph feels natural.

The advanced tools, `git bisect` for finding which commit introduced a bug, `git rebase -i` for cleaning up history before a pull request, `git subtree` For managing shared code across repos, these will all make immediate sense because you understand the underlying graph.

“Git doesn’t have an undo button. It has a graph. Once you see the graph, you are never stuck.”

The fear around Git comes from not knowing what the commands are doing to the graph. Now you know. The rest is just practice.

Artificial Intelligence

Data Science

Software Development

Technology

Programming



Published in AI Engineering Simplified

Follow

70 followers · Last published 7 hours ago

Deep dives into how real AI systems work and how to build systems that work in production. The aim is to explain complex Ai topics in a simplified and practical manner.



Written by Divy Yadav

Follow

1.6K followers · 39 following

I write about how real AI systems work and how to build production-ready, resilient AI applications simply and practically.
<https://github.com/dvy246>

No responses yet



Pancake Agree

What are your thoughts?

-
- Help

Status

About

Careers

Press

Blog

Privacy

Rules
- Terms

Text to speech